

Reviewer Pack — Minimal Rank of Topological Insulator Indices in Quantum Que...

Entry 76d3eff0508f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 15:42:48 UTC

Minimal Rank of Topological Insulator Indices in Quantum Query Complexity

Entry ID: 76d3eff0508f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 15:42:48 UTC

1. Conjecture

Field A (mathematical branch): Topological Insulators and Symmetry Properties **Field B** (complexity object): Quantum Computing: Query Complexity

Statement:

[‘For every quantum query algorithm Q computing a Boolean function f with query complexity q , the minimal rank of the topological insulator index for the associated Berry phase is $O(q)$.’, ‘Equivalently, there exists a constant $c > 0$ such that for all quantum query algorithms Q , the minimal rank of the topological insulator index for the associated Berry phase satisfies $\min_rank(\text{Berry_phase}(Q)) \leq c \cdot q$.’, ‘For a quantum query algorithm Q , let $\text{Berry_phase}(Q)$ be the first nonzero eigenvalue of its unitary representation U in the space of quantum states, and denote by $T(Q)$ the minimal rank of its topological insulator index. Then, for all $q \geq 1$, $T(Q) = O(q)$.’]

Rationale (proposer’s reasoning):

[‘Topological insulators exhibit robust properties against perturbations, which can be characterized by their Berry phases. These phases are closely related to quantum algorithms through the concept of unitary representations.’, ‘The minimal rank of the topological insulator index provides a measure of the complexity of the associated Berry phase, and thus a potential connection to quantum query complexity.’, ‘This conjecture suggests that the inherent complexity of a quantum algorithm can be captured by its topological properties, which has not been previously explored in the context of complexity theory.’]

Taxonomy category: Topological_Insulators (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1e8440bb53680fe7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all quantum query algorithms Q with varying query complexities q , the ratio between the minimal rank of the topological insulator index ($T(Q)$)

and the query complexity (q) remains below a constant $c \leq 1$ when computed over 30 random seeds. The conjecture is falsified if any seed produces a ratio $T(Q)/q$ exceeding c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "topological insulators" AND "quantum query complexity" - "Berry phase" AND unitary representation IN quantum computing" - "minimal rank" OF topological insulator index AND $O(q)$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def unitary_representation(f):
        n = int(math.log2(len(f)))
        U = [[0] * len(f) for _ in range(len(f))]
        for i in range(len(f)):
            for j in range(len(f)):
                if f[i] == f[j]:
                    U[i][j] = 1
        return U

    def berry_phase(U):
        n = int(math.log2(len(U)))
        eigenvalues = [0] * len(U)
        for i in range(n):
            for j in range(i + 1, n):
                if U[i][j] != U[j][i]:
                    eigenvalues[i] += 1
                    eigenvalues[j] -= 1
```

```

    return max(eigenvalues)

def minimal_rank(Berry_phase):
    rank = 0
    for i in range(len(Berry_phase)):
        if Berry_phase[i] != 0:
            rank += 1
    return rank

n = random.randint(5, 40)
f = generate_boolean_function(n)
U = unitary_representation(f)
Berry_phase = berry_phase(U)
rank = minimal_rank(Berry_phase)

metric_value = rank / n if n > 0 else float('inf')
conjecture_holds = metric_value <= 1
counterexample = "" if conjecture_holds else f"rank={rank}, expected<=1"

return {
    "metric_name": "minimal_rank",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, ...{result}...}}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\\"{results[seeds.index(first_failing_seed)]['counterexample']}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: Re-run the test with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12538
2	preregistration	ollama_remote	glm4:latest	0	0	5916
3	novelty	ollama_remote	glm4:latest	0	0	4735
4	novelty	ollama_remote	glm4:latest	0	0	5422
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13590
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20672
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10230
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9640
9	judge	ollama_remote	glm4:latest	0	0	45910

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128652 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/76d3eff0508f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/76d3eff0508f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/76d3eff0508f.tar.gz (if generated)